UNIT-III

LECTURE 1

Distributed Process Scheduling

Distributed scheduling (DS) is an approach that enables local decision makers to create schedules that consider local objectives and constraints within the boundaries of the overall system objectives.

Communication and synchronization facilities are essential system components for supporting concurrent execution of interacting processes. Before execution, processes need to be scheduled and allocated with resources.

The primary objective of scheduling is to enhance overall system performance matrices such as process completion time and processor utilization.

A system performance model:

Parallel and distributed algorithms are represented by set of multiple processes governed by rules that regulate the interactions among processes.

- Precedence processes model
- Communication process model
- Disjoint process model

Partitioning a task into multiple processes for execution can result in a **speedup** of the total task completion time. The speedup factor S is a function

$$S = F(Algorithm, System, Schedule)$$

S can be written as:

$$S = \frac{OSPT}{CPT} = \frac{OSPT}{OCPT_{ideal}} \times \frac{OCPT_{ideal}}{CPT} = S_i \times S_d$$

where

- $OSPT$ = optimal sequential processing time
- CPT = concurrent processing time
- $OCPT_{ideal}$ = optimal concurrent processing time
- S_i = the ideal speedup
- S_d = the degradation of the system due to actual implementation compared to an ideal system

S_i can be rewritten as:

$$S_i = \frac{RC}{RP} \times n$$

where

$$RP = \frac{\sum_{i=1}^{m} P_i}{OSPT}$$

and

$$RC = \frac{\sum_{i=1}^{m} P_i}{OCPT_{ideal} \times n}$$

and n is the number of processors. The term $\sum_{i=1}^{m} P_i$ is the total computation of the concurrent algorithm where m is the number of tasks in the algorithm. S_d can be rewritten as:

$$S_d = \frac{1}{1+\rho}$$

where

$$\rho = \frac{CPT - OCPT_{ideal}}{OCPT_{ideal}}$$

RP is **Relative Processing**: how much loss of of speedup is due to the substitution of the best sequential algorithm by an algorithm better adapted for concurrent implementation. RC is the **Relative Concurrency** which measures how far from optimal the usage of the n-processor is. It reflects how well adapted the given problem and its algorithm are to the ideal n-processor system. The final expression for speedup S is

$$S = \frac{RC}{RP} \times \frac{1}{1+\rho} \times n$$

The term ρ is called *efficiency loss*. It is a function of scheduling and the system architecture. It would be decomposed into two independent terms: $\rho = \rho_{sched} + \rho_{syst}$, but this is not easy to do since scheduling and the architecture are interdependent. The best possible schedule on a given system hides the communication overhead (overlapping with other computations).

The **unified speedup model** integrates three major components

- algorithm development
- system architecture
- scheduling policy

LECTURE 2

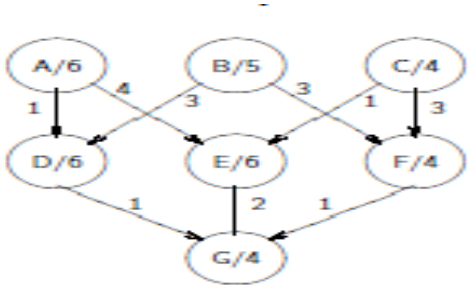
Static Process Scheduling:

- Scheduling a set of partially ordered tasks on a nonpreemtive multiprocessor system of identical processors to minimize the overall finishing time (makespan)
- Except for some very restricted cases scheduling to optimize makespan is **NP-complete**
- Most research is oriented toward using approximate or heuristic methods to obtain a *near optimal* solution to the problem
- A good heuristic distributed scheduling algorithm is one that can best balance and overlap computation and communication

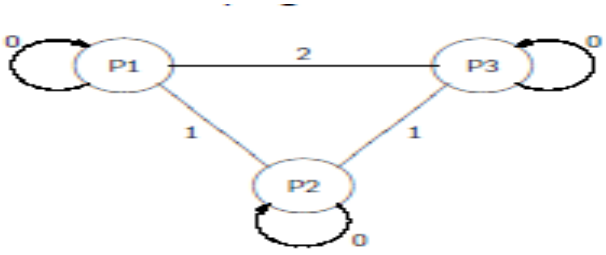
In static scheduling, the mapping of processes to processors is determined before the execution of the processes. Once a process is started, it stays at the processor until completion.

Precedence process model:

- Program is represented by Directed Acyclic Group(DAG)
- Computational Model
- Primary objective of task scheduling is to achieve maximal concurrency for task execution with in program



(a) Precedence process model

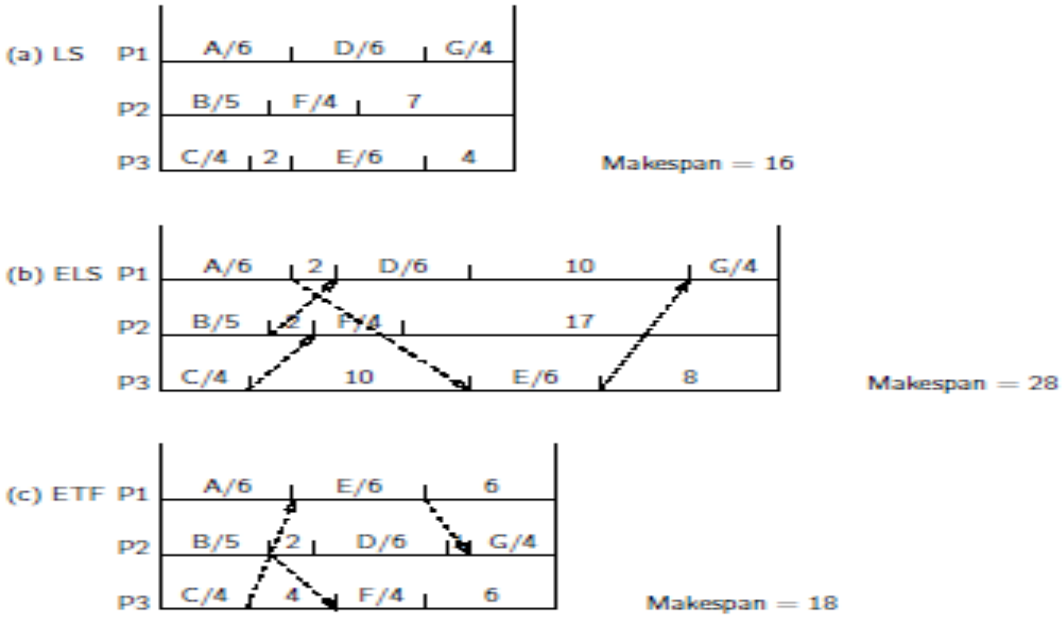


(b) Communication system model

Finding the minimum makespan is NP-complete, so we will rely on heuristic algorithms for finding good mapping of the process model to the system model. For precedence process graphs, the notion of **critical path** is useful - the longest execution path in the DAG, which is the lower bound of the makespan. Simple heuristic: map all tasks in a critical path onto a single processor.

1. **List Scheduling** (LS) strategy: No processor remains idle if there are some tasks available that it could process (without considering communication overhead).

2. **Extended List Scheduling** (ELS) strategy: Allocating tasks to processors according to LS and adding communication delays. communication overhead.
3. **Earliest Task First** (ETF): The earliest schedulable task is scheduled first (calculation includes communication overhead).



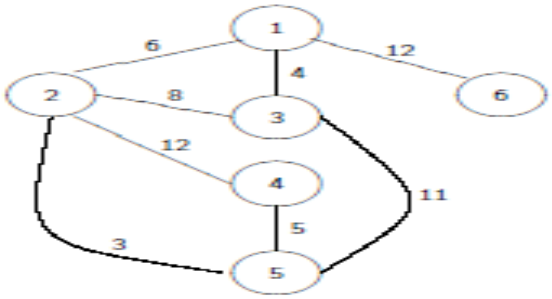
Communication process model

LECTURE 3

- Process scheduling for many system applications has a perspective very different from precedence model - applications may be created independently, processes do not have explicit completion time and precedence constraints
- Primary objectives of process scheduling are to maximize resource utilization and to minimize interprocess communication
- Communication process model is an undirected graph G with node and edge sets V and E, where nodes represent processes and the weight on an edge is the amount of interaction between two connected processes

Process	Cost on A	Cost on B
1	5	10
2	2	Infinity
3	4	4
4	6	3
5	5	2
6	Infinity	4

(a) Computation set



(b) Communication cost

Objective function called **Module Allocation** for finding an optimal allocation of m process modules to P processors:

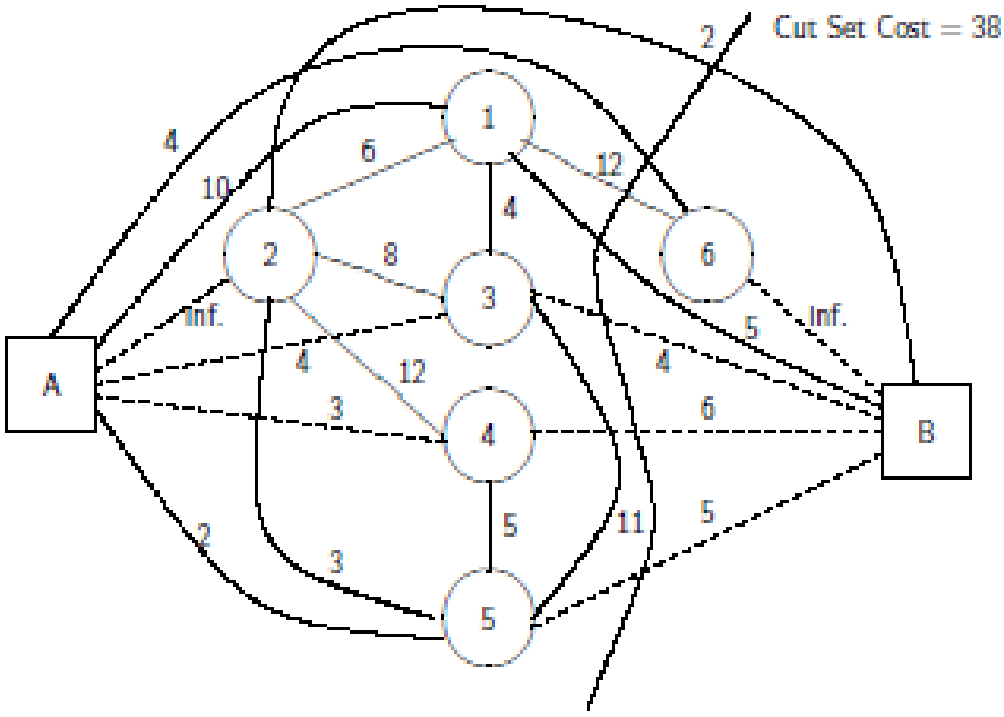
$$Cost(G, P) = \sum_{j \in V(G)} e_j(p_i) + \sum_{(i,j) \in E(G)} c_{i,j}(p_i, p_j)$$

where $e_j(p_i)$ is the execution cost of process j on p_i, which is the processor allocated to process j and $c_{i,j}(p_i, p_j)$ is the communication cost between two processes i and j, allocated to two different processors p_i and p_j.

For $P = 2$ there is an efficient polynomial-time solution using Ford-Fulkerson's maximum-flow algorithm - equivalent to the *minimum cut* set in the graph.

Heuristic solution: separate optimization of computation and communication into two independent phases.

- Processes with higher interprocess interaction are merged into clusters
- Each cluster is then assigned to the processor that minimizes the computation cost



LECTURE 4

Dynamic load sharing and balancing:

The assumption of prior knowledge of processes is not realistic for most distributed applications. The disjoint process model, which ignores the effect of the interdependency among processes, is used. Objective of scheduling: *utilization* of the system (has direct bearing on throughput and completion time) and *fairness* to the user processes (difficult to define).

If we can designate a controller process that maintains the information about the queue size of each processor:

- Fairness in terms of equal workload on each processor (join the shortest queue) - migration workstation model (use of **load sharing** and **load balancing**, perhaps **load redistribution** i.e. **process migration**)
- Fairness in terms of user's share of computation resources (allocate processor to a waiting process at a user site that has the least share of the processor pool) - processor pool model

Solutions without a centralized controller: **sender-** and **receiver-initiated** algorithms.

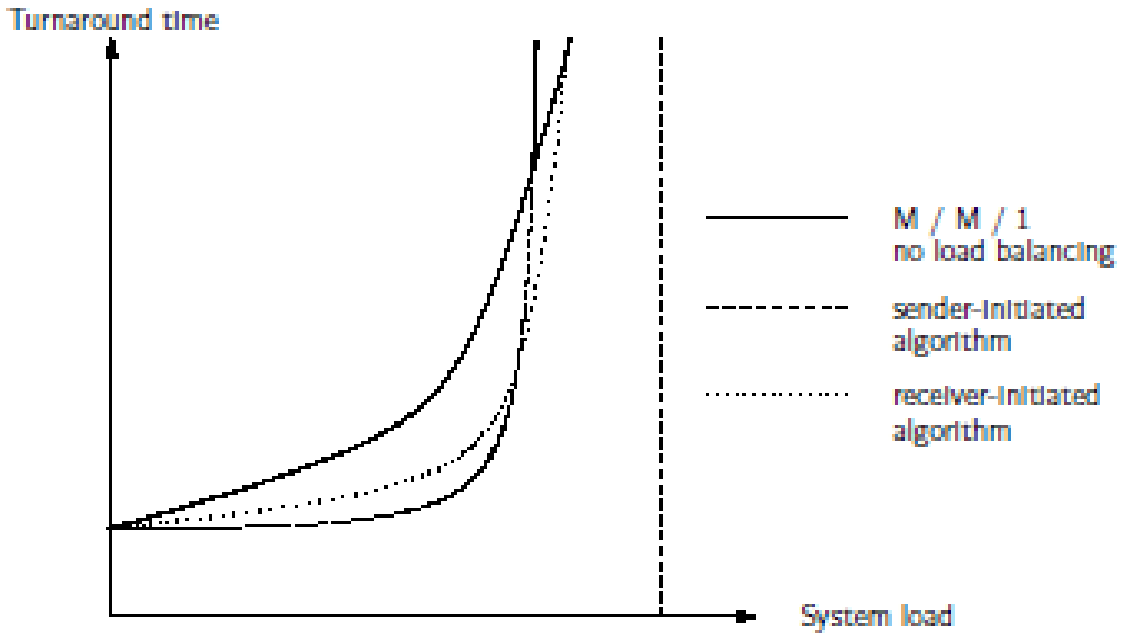
Sender-initiated algorithm:

- Push model
- Includes probing strategy for finding a node with the smallest queue length(perhaps multicast)
- Performs well on a lightly loaded system

Receiver-initiated algorithm:

- Pull model
- Probing strategy can also be used
- More stable
- Perform on average better

Combinations of both algorithms are possible: choice based on the estimated system load information or reaching threshold values of the processing node's queue.



Performance comparison of dynamic load-sharing algorithms

Distributed file system

File system - storage for persistent named data objects (files); responsible for creation, deletion, retrieval, modification and protection of files.

Distributed file system (DFS) is an implementation of a file system that consists of physically dispersed storage sites but which provide a traditional file system view for users (files crossing system boundaries is transparent to the users). Many important concepts in distributed systems design can be demonstrated by the implementation of a DFS:

1. DFS employ many aspects of transparency
2. Directory service in a DFS is a good example of a name service
3. Use of caching and replication → problem of cache coherence and replica management
4. Access control and protection → distributed system security problems

Characteristics of a DFS:

Two key characteristics: *dispersion* and *multiplicity* of users and files. Goal: hiding this from users. A transparent DFS should exhibit the following transparency properties:

- **Dispersed clients:** *login transparency* and *access transparency* (regardless of whether files are on the local or remote host)
- **Dispersed files:** names used to refer to files need not contain information about their physical location - *location transparency*, *location independence* (can be moved from one location to another without changing their names)
- **Multiplicity of users:** *concurrency transparency*, desirable to support at the transaction level
- **Multiplicity of files:** *replication transparency* (atomic updates on the replicas)

Other desirable characteristics: fault tolerance, scalability, heterogeneity of the system.

DFS design and implementation:

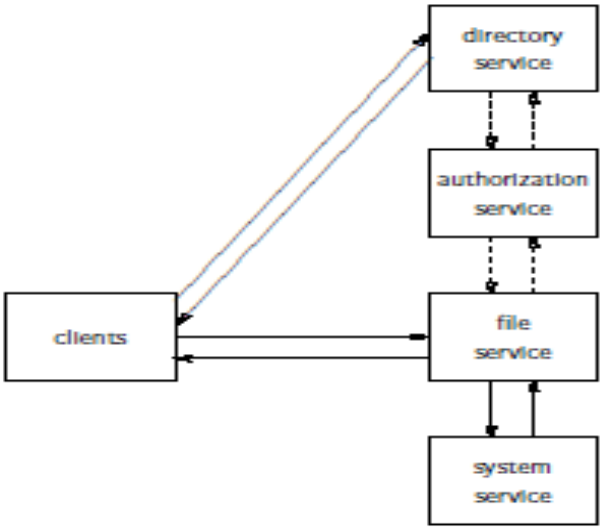
To the users, a file consists of: **file name**, **file attributes**, **data units**. Organization of data units: *flat* or *hierarchical*. File accesses:

- *sequential access*: file position is maintained with each open file - used in a *session* of subsequent file accesses
- *direct access*: explicit reference fixed-size data units by their block number
- *indexed sequential access*: data units addresses directly using an index (key) associated with each data block

Major components of a file system:

directory service		name resolution, addition and deletion of files
authorization service		capability and/or access control list
file service	transaction	concurrency and replication management
	basic	read/write files and get/set attributes
system service		device, cache, and block management

Services in a DFS → usually structured to provide directory, authorization, file and system services separately.



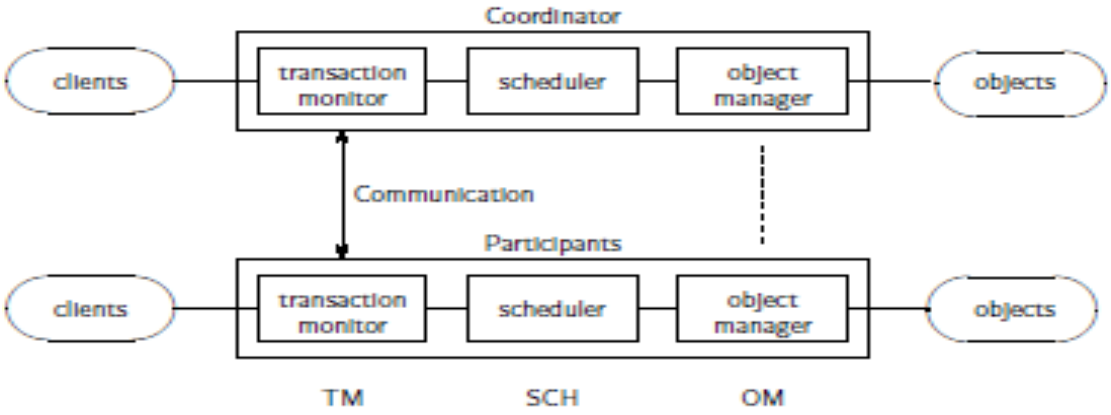
File mounting and server registration

Directories are hierarchically structured. Mounting operation: attaches a remote named file system to the client's file system hierarchy at the position pointed to by a path name (usually a leaf of the directory tree that contains only an empty subdirectory). Linkage information (*mount table*) for the remote FS is maintained by the host FS. A file server normally restricts

LECTURE 5

Transaction and concurrency control:

Simple architecture for a distributed transaction processing system:



Atomicity is the primary goal of atomic transactions. Isolation and durability are secondary in the sense that they are needed only to support atomicity.

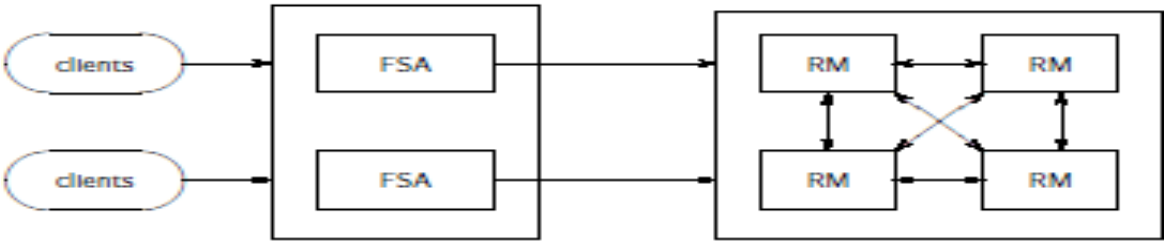
Optimistic concurrency control:

- Allowing an entire transaction to complete and the validate the transaction before making its effect permanent
- Assumes a private workspace that contains shadow copies of the shared objects at each transaction site
- In the execution phase no validation of consistency is performed
- On end transaction the TM begins the validating phase – a transaction is globally validated among TMs using a two-phase commit protocol
- After the consistency of the transaction is validated, the TM enters the update phase to make changes permanent in the persistent memory.
- Once a transaction is validated, it is guaranteed to be committed.

Data and file replication

Updates to all replicas must be *atomic*. **One-copy serializability** for distributed transaction services: combined atomic transaction and atomic update property.

Architecture for management of replicas



Object access operations: *read*, *update* (*overwrite*, *read-modify*).

Options for read operations:

- read-one-primary
- read-one
- Read-quorum

Options for write operations:

- Write-one-primary
- Write-all
- Write-all-available
- Write-quorum

LECTURE 6

CASE STUDY

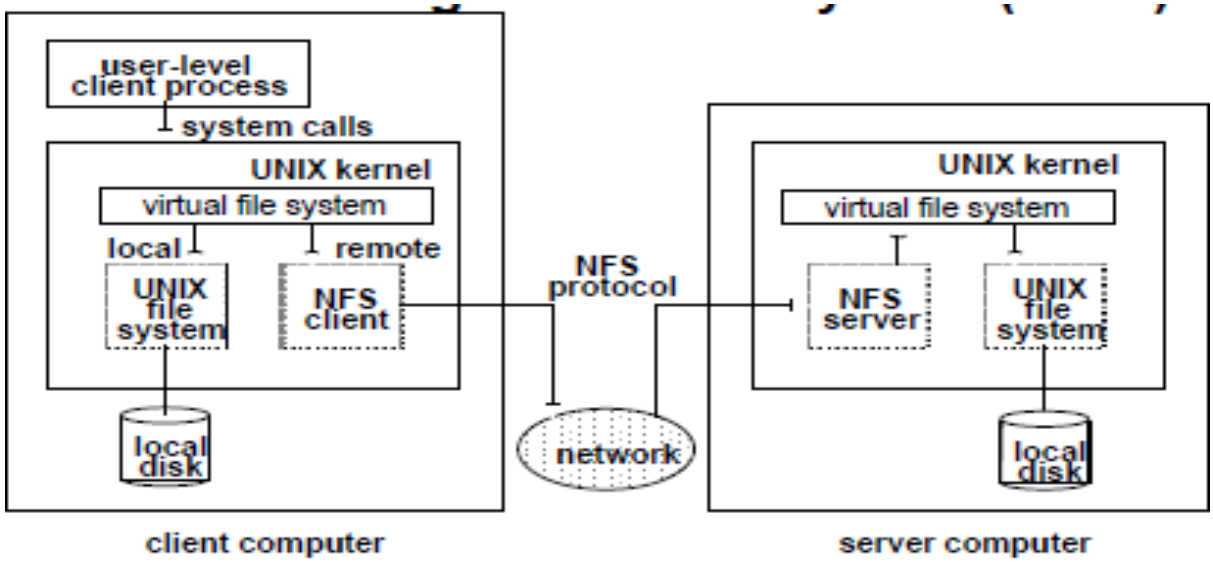
1. NFS(SUN)
2. GPFS
3. GENERAL FILE SYSTEM (ANDREW AND CODA FILE)

NFS (SUN)

Sun's network file systems (NFS)

- Designed by Sun Microsystems
  - First distributed file service designed as a project, introduced in 1985
  - To encourage its adoption as a standard
    - Definitions of the key interfaces were placed in the public domain in 1989
    - Source code for a reference implementation was made available to other computer vendors under license
    - Currently the de facto standard for LANs
- Provides transparent access to remote files on a LAN, for clients running on UNIX and other operating systems
  - A UNIX computer typically has a NFS client and server module in its OS kernel
    - Available for almost any UNIX
    - Client modules are available for Macintosh and PCs

- Remote file systems may be
  - Hard mounted — when a user-level process accesses a file, it is suspended until the request can be completed
    - If a server crashes, the user-level process will be suspended until recovers
  - Soft mounted — after a small number of retries, the NFS client returns a failure code to the user process
    - Most UNIX utilities don't check this code…
- Automounting
  - The automounter dynamically mounts a file system whenever an "empty" mount point is referenced by a client
    - Further accesses do not result in further requests to the automounter…
    - Unless there are no references to the remote file system for several minutes, in which case the automounter unmounts it



- Virtual file system:
  - Separates generic file-system operations from their implementation (can have different types of local file systems)
  - Based on a file descriptor called a vnode that is unique networkwide (UNIX inodes are only unique on a single file system)

LECTURE 7

NFS protocol

- NFS protocol provides a set of RPCs for remote file operations
  - Looking up a file within a directory
  - Manipulating links and directories
  - Creating, renaming, and removing files
  - Getting and setting file attributes
  - Reading and writing files
- NFS is stateless
  - Servers do not maintain information about their clients from one access to the next
    - There are no open-file tables on the server
  - There are no open and close operations
    - Each request must provide a unique file identifier, and an offset within the file
  - Easy to recover from a crash, but file operations must be idempotent

- Because NFS is stateless, all modified data must be written to the server's disk before results are returned to the client
  - Server crash and recovery should be invisible to client —data should be intact
  - Lose benefits of caching
    - Solution — RAM disks with battery backup (un-interruptable power supply), written to disk periodically
- A single NFS write is guaranteed to be atomic, and not intermixed with other writes to the same file
  - However, NFS does not provide concurrency control
    - A `write` system call may be decomposed into several NFS writes, which may be interleaved
    - Since NFS is stateless, this is not considered to be an NFS problem

Andrew file system (AFS)

- Designed by Carnegie Mellon University
  - Developed during mid-1980s as part of the Andrew distributed computing environment
  - Designed to support a WAN of more than 5000 workstations
  - Much of the core technology is now part of the Open Software Foundation (OSF) Distributed Computing Environment (DCE), available for most UNIX and some other operating systems
- AFS was made to span large campuses and scale well therefore the emphasis was placed on offloading the work to the clients
- as much as possible data is cached on clients, uses session semantics - cache consistency operations are done when file is opened or closed
- Provides transparent access to remote files on a WAN, for clients running on UNIX and other operating systems
  - Access to all files is via the usual UNIX file primitives
  - Compatible with NFS — servers can mount NFS file systems

- AFS is stateful, when a client reads a file from a server it holds a callback, the server keeps track of callbacks and when one of the clients closes the file (and synchronizes it's cached copy) and updates it, the server notifies all the callback holders of the change breaking the callback, callbacks can be also broken to conserve storage at server
- problems with AFS:
  - even if the data is in local cache - if the client performs a write a complex protocol of local callback verification with the server must be used; cache consistency preservation leads to deadlocks
  - in a stateful model, it is hard to deal with crushes.

LECTURE 8

Caching in Andrew

- When a remote file is accessed, the server sends the entire file to the client
  - The entire file is then stored in a disk cache on the client computer
    - Cache is big enough to store several hundred files
- Implements session semantics
  - Files are cached when opened
  - Modified files are flushed to the server when they are closed
  - Writes may not be immediately visible to other processes
- When client caches a file, server records that fact — it has a callback on the file
  - When a client modifies and closes a file, other clients lose their callback, and are notified by server that their copy is invalid

How can Andrew perform well?

- Most file accesses are to files that are infrequently updated, or are accessed by only a single user, so the cached copy will remain valid for a long time
- Local cache can be big — maybe 100 MB — which is probably sufficient for one user's working set of files
- Typical UNIX workloads:
  - Files are small, most are less than 10kB
  - Read operations are 6 times more common than write operations
  - Sequential access is common, while random access is rare
  - Most files are read and written by only one user; if a file shared, usually only one user modifies it
  - Files are referenced in bursts

AFS vs. NFS

- NFS is simpler to implement, AFS is cumbersome due to cache consistency synchronization mechanisms, etc
- NFS does not scale well - it is usually limited to a LAN, AFS scales well - it may span the Internet
- NFS performs better than AFS under light to medium load. AFS does better under heavy load
- NFS does writes faster(?)

what is the available operation on AFS (ANDREW FILE SYSTEM)?

ANS:-

1. LOOKUP:- ALLOWS A USER TO LIST THE CONTENT OF THE AFS DIRECTORY.
2. INSERT:- ALLOW A USER TO ADD NEW FILE OR SUBDIRECTORIES TO THE DIRECTORY.
3. DELETE:- ALLOW A USER TO DELETE FILE OR SUBDIRECTORIES FROM THE DIRECTORY.
4. ADMINISTRATER:- ALLOW A USER TO CHANGE THE PERMISSION OF THE AFS DIRECTORY(HOME).
5. READ:-ALLOW A USER TO LOOK AT THE CONTENTS OF FILE AND SUBDIRECTORIES TO THE AFS DIRECTORIES, UNTIL READ ACCESS TO ANY USER.
6. WRITE:-ALLOW A USER TO MODIFY FILES IN A DIRECTORY.

HOW DOES AFS WORK?

CODA (CONSTANT DATA AVAILABILTY) FILE SYSTEM:

Coda is a file system for a large-scale distributed computing environment composed of Unix workstations. It provides resiliency to server and network failures through the use of two distinct but complementary mechanisms.

1. One mechanism, ***server replication***, stores copies of a file at multiple servers.

2. The other mechanism, ***disconnected operation***, is a mode of execution in which a caching site temporarily assumes the role of a replication site. Disconnected operation is particularly useful for supporting portable workstations. The design of Coda optimizes for availability and performance, and strives to provide the highest degree of consistency attainable in the light of these objectives. Measurements from a prototype show that the performance cost of providing high availability in Coda is reasonable.

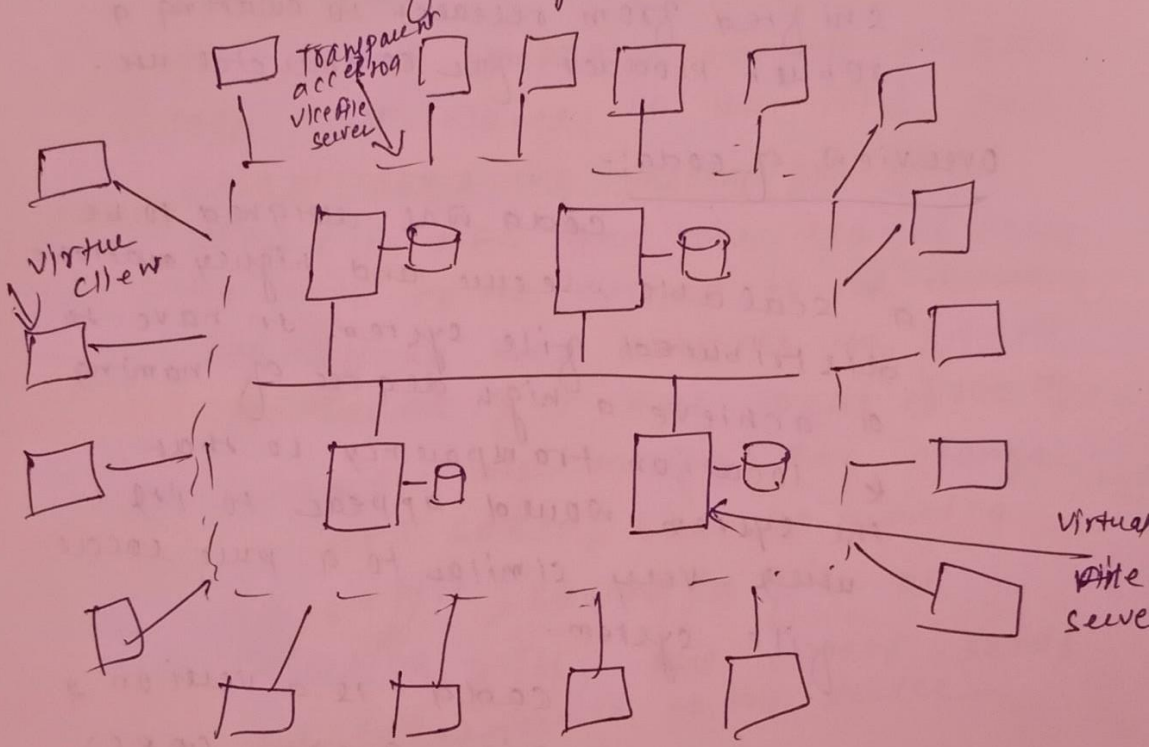
3. IT WAS DESIGNED TO BE A SCALABLE, SECURE, AND HIGHLY AVAILABLE DISTRIBUTED FILE SYSTEM.

4. CODA IS A 2ND VERSON OF ANDREW FILE SYSTEM(AFS) WITH THE SAME ORGANISATION.

The *Coda File System* is a descendant of AFS that is substantially more resilient to failures. The ideal that Coda strives for is *constant data availability*, allowing a user to continue working regardless of failures elsewhere in the system. Our goal is to provide users with the benefits of a shared data repository, but to allow them to rely entirely on local resources when that repository is partially or totally inaccessible. A related goal of Coda is to gracefully integrate the use of

AFS with portable computers. At present, users manually copy relevant files from AFS, use the machine while isolated from the network, and manually copy updated files back to AFS upon reconnection. These users are effectively performing manual caching of files with write-back on reconnection. If one views the disconnection from AFS as a deliberately induced failure, it is clear that a mechanism for supporting portable machines in isolation is also a mechanism for fault tolerance.

→ Coda was designed to be scalable, secure; and highly available distributed file system. An important goal was to achieve a high degree of naming and location transparency so that system would appear to its use very similar to a pure local file system



FEATURES OF CODA FILE SYSTEM:-

1. THROUGH THE SERVER REPLICATION DATA AVAILABILITY IS INCREASED.
2. SECURITY MODEL FOR AUTHENTICATION, ENCRYPTION AND ACCESS CONTROL
3. GOOD SCALABILITY
4. HIGH PERFORMANCE THROUGH THE CLIENT SIDE FROM THE CACHE CONSISTENCY
5. CONTINUE OPERATION DURING PARTIAL NETWORK FAILURES IN SERVER NETWORK.
6. SHARING IS WELL DEFINED IN THE PRESENCE OF NETWORK FAILURE.

- FOR THE TRANSPARENCY THE SYSTEM WOULD APPEAR TO ITS USER TO A PURE LOCAL FILE SYATEM.
- FEATURES OF CODA FILE SYSTEM
    1. HIGH PERFORMANCE THROUGH THE CLIENT SIDE
    2. SERVER REPLICATION (THROUGH THE REPLICATION MANAGER)
    3. IN THIS AUTHENTICATION ENCRYPTION AND ACCESS CONTROL IS USED.
    4. CONTINUE OPERATION DURING PARTIAL NETWORK FAILURE IN SERVER NETWOK(BECAUSE OF SERVER REPLICATION)
    5. GOOD SCALABILITY
    6. DURING PARTIAL NETWORK FAILURE WELL DEFINED FOR SHARING OF FILE ETC.

Features :-

- Disconnected operation for mobile computing.
- Is freely available under a liberal licence
- High performance through client side persistent caching.
- Server replication
- Security model for authentication, encryption and access control.
- Continued operation during partial network failures in server n/w.
- Good scalability
- Well defined semantics of sharing, even in the presence of n/w failures